

Study Plan — Step by Step

Goal: finish **CMU 15-445/645 Fall 2025** (all 24 lectures, homeworks 1–4, projects 0–2), this repo's OS, Networking, Version Control, System Design, Programming Languages and Concepts, and System Architecture domains, contest practice, a small career track, and the **flagship project** (Raft KV store in Rust; full guide at the end of this PDF).

How it works: no dates. Do the **lowest unfinished step**, tick it, then move to the next. Time during military service varies, so the pace is open; only the order matters. **Friday** (the one day off) is for the long steps: 2–4h project chunks, virtual contests, flagship coding.

Out of scope: repo domains 1 (Data Structures), 2 (Algorithms) and 9 (Problem-solving and Coding). Contest-Practice stays.

The only exceptions: real deadlines

If one of these dates gets close, do that step first, then return to the sequence.

- Step 14: Knight-Hennessy — go/no-go (optional stretch) — **2026-10-06**
- Step 19: DAAD: document checklist + programme shortlist — **2026-10-30**
- Step 24: Book the IELTS test (needed for master's / Canada) — **2026-10-15**
- Step 34: DAAD motivation letter — first draft — **2026-10-30**
- Step 46: DAAD motivation letter — final; upload documents — **2026-10-30**
- Step 52: Submit the DAAD application — **2026-10-30**
- Step 137: ETH Zurich Excellence Scholarship (ESOP) — apply (window 1–30 Nov) — **2026-11-30**

Phases and checkpoints

Phase	Steps	Checkpoints
1 · Setup + SQL + P0	1–19	Step 10: P0 passes on Gradescope · Step 16: HW1 self-graded
2 · Storage + HW2	20–29	Step 24: IELTS booked · Step 26: HW2 self-graded
3 · P1 Buffer Pool	30–61	Step 52: DAAD submitted · Step 59: P1 passes on Gradescope · Step 60: flagship repo created
4 · Indexes + execution	62–76	Step 68: HW3 self-graded
5 · P2 B+Tree + concurrency	77–96	Step 96: Raft election runs in simulation
6 · Distributed DBs + wrap-up	97–104	Step 98: HW4 · Step 101: all 24 lectures · Step 102: P2 passes · Step 103: flagship M0 done
7 · Next	105–138	CMU P3/P4 + HW5–6 (105, 111, 121) alternating with every guide of domain 7 Programming Languages and Concepts (106–120, incl. design patterns), domain 8 System Architecture (122–129) and the rest of System Design (130–132); then contests, mock interviews, final sweep. Alternate with flagship M1–M8.

Rules

- Hours are estimates. If a step takes longer, that's fine; don't skip ahead.
- Never skip a project checkpoint (P0, P1, P2 on Gradescope).
- Short on time? Do a short step (a guide, a career step, a reading); the long ones wait for Friday.
- After step 104, alternate one Phase 7 step with one flagship milestone (M1 → M8).
- Keep your BusTub copy **private** (course academic-integrity policy).

Time by track (phases 1–6)

CMU Project	29h
CMU Lecture	24h
Flagship Project	20h
Career	9h
Repo	8h
CMU Homework	8h
Contest Practice	4.5h
Review	1.5h
Total	104h

Key links

- CMU 15-445 Fall 2025: [schedule](#) · [assignments](#) · [FAQ](#) · [lecture playlist](#) · Gradescope for non-CMU students: code **5R4XPZ**, school "Carnegie Mellon University"
- Repo: [README checklist](#) · [Contest PDFs](#) · [Application tracker](#) · [Study-Plan folder](#)

Phase 1 · Setup + SQL + P0					steps 1–19 · ≈20h
☐	1	CMU Lecture	L01 — Relational Model & Algebra Reading: Chapters 1-2 Tick: Relational databases (SQL)	1h	Video · Slides · Notes
☐	2	CMU Lecture	L02 — Modern SQL Reading: Chapters 3-5 Tick: DQL, DML, DDL, Joins, Views	1h	Video · Slides · Notes
☐	3	CMU Project	Setup: clone BusTub into a private repo, build (CMake + clang), gdb ready; join non-CMU Gradescope (code 5R4XPZ, school: Carnegie Mellon University) Follow "Creating Your Own Project Repository" + "Setting Up Your Development Environment" on the P0 page	1h	Project spec
☐	4	CMU Project	P0 C++ Primer — Task #1 (first pass)	2.5h	Project spec
☐	5	Repo	OS: Processes + Threads guides Tick: Operating Systems → Processes; Threads	1h	Guide · Threads guide
☐	6	Flagship Project	Flagship: Raft paper §1–§5 + play with the raft.github.io visualization	1h	Raft paper · Visualization
☐	7	Career	Update LinkedIn to match the new CVs (headline, GSoC 2026, open source, projects) The 3 CV versions are already done (CV folder).	0.5h	CV folder
☐	8	CMU Lecture	L03 — Database Storage I Reading: Chapter 12.1-12.4, 13.2-13.3	1h	Video · Slides · Notes
☐	9	CMU Lecture	L04 — Memory Management Reading: Chapter 13.2-13.5	1h	Video · Slides · Notes
☐	10	CMU Project	P0 — finish, run tests + make check-clang-tidy + memory-leak check, submit to Gradescope (make submit-p0) Milestone: P0 passes on Gradescope	1.5h	Project spec
☐	11	CMU Homework	HW1 SQL (DuckDB) — Q1–Q4 Tick: Databases → DQL / Joins	1.5h	Homework
☐	12	Flagship Project	Flagship: Raft paper §6–§8; write your own one-page Figure 2 cheat sheet Then read the Students' Guide to Raft	1h	Raft paper · Students' Guide to Raft
☐	13	Contest Practice	Contest PDF #1: CF Round 1074 (Div. 4) — virtual, solve A–D, then read hints	1.5h	Contest PDFs
☐	14	Career	Knight-Hennessy — go/no-go (optional stretch)  Deadline 2026-10-06 Hard deadline: do it before the date even if earlier steps are not finished.	0.5h	Roadmap file
☐	15	CMU Lecture	L05 — Database Storage II Reading: Chapter 14.8.1, 24.2	1h	Video · Slides · Notes
☐	16	CMU Homework	HW1 — Q5–Q6, then compare with the official solutions	1h	Homework · Solutions
☐	17	Repo	OS: Thread synchronization (mutexes, semaphores, monitors) Tick: Operating Systems → Thread synchronization mechanisms (Mutexes, Semaphores, Monitors)	0.5h	Guide
☐	18	Flagship Project	Flagship: DDIA ch. 9 part 1 — linearizability, ordering guarantees	1h	DDIA book
☐	19	Career	DAAD: document checklist + programme shortlist  Deadline 2026-10-30 Hard deadline 30 Oct: the DAAD steps (19, 34, 46, 52) must be done before it.	0.5h	Roadmap file

Phase 2 · Storage + HW2					steps 20–29 · ≈8h
☐	20	CMU Lecture	L06 — Storage Models & Compression Reading: Chapter 11.2, 13.6	1h	Video · Slides · Notes
☐	21	CMU Homework	HW2 Storage — first half	1h	Homework
☐	22	Repo	OS: Scheduling algorithms Tick: Operating Systems → Scheduling algorithms (Fair share, EDF, WFQ)	0.5h	Guide
☐	23	Flagship Project	Flagship: DDIA ch. 9 part 2 — consensus; Jepsen consistency-models map	1h	Jepsen models
☐	24	Career	Book the IELTS test (needed for master's / Canada)  Deadline 2026-10-15 Pick a test date that fits your service schedule.	0.5h	Roadmap file

☐	25	CMU Lecture	L07 — Hash Tables Reading: Chapter 14.5, 24.5	1h	Video · Slides · Notes
☐	26	CMU Homework	HW2 Storage — finish + self-grade with the official solutions	1h	Homework · Solutions
☐	27	Repo	OS: Memory management (page replacement, mmap, buddy allocation) Page replacement (LRU, Clock) is the same idea as the CMU P1 replacer. Tick: Operating Systems → Memory management (Page replacement, Memory-mapped files, Buddy allocation)	0.5h	Guide
☐	28	Flagship Project	Flagship: DDIA ch. 5 — replication (leaders, logs, lag)	1h	DDIA book
☐	29	Career	Shortlist 5 P1 companies from the application tracker	0.5h	Roadmap file

Phase 3 · P1 Buffer Pool				steps 30–61 · ≈32h	
☐	30	CMU Project	P1 Buffer Pool Manager — read the spec; work through the ARC algorithm on paper	1h	Project spec
☐	31	Repo	Databases domain: read SQL-DQL, SQL-DML, Joins, Normalization guides; tick what HW1 + L01–L02 covered Tick: Databases → Relational databases (SQL) sub-items	1h	Guide
☐	32	Repo	OS: Deadlock detection and prevention Tick: Operating Systems → Deadlock detection and prevention	0.5h	Guide
☐	33	Flagship Project	Flagship: DDIA ch. 6 — partitioning (range vs hash, rebalancing)	1h	DDIA book
☐	34	Career	DAAD motivation letter — first draft  Deadline 2026-10-30	0.5h	Roadmap file
☐	35	CMU Project	P1 Task #1 — ARC replacer: implementation start	1.5h	Project spec
☐	36	Repo	OS: File systems; tick all finished OS checklist items Tick: Operating Systems → File systems (Journaling, NFS/SMB, Encryption, HDFS/Ceph)	0.5h	Guide
☐	37	Flagship Project	Flagship: Tokio tutorial part 1 (spawning, shared state)	1h	Tokio tutorial
☐	38	Career	Send 1 application (P1 list) with the matching CV version	0.5h	Roadmap file
☐	39	Review	Checkpoint review #1: tick finished checklist items in the repo README; re-plan anything that slipped	0.5h	README checklist
☐	40	CMU Lecture	L08 — Indexes & Filters I Reading: Chapter 14.1–14.4 Tick: Indexing → B-tree, B+ tree	1h	Video · Slides · Notes
☐	41	CMU Lecture	L09 — Indexes & Filters II Reading: Chapter 14.1–14.4, 24.1 Tick: Indexing → Bitmap indexing	1h	Video · Slides · Notes
☐	42	CMU Project	P1 Task #1 — ARC replacer: finish + pass local tests	2h	Project spec
☐	43	CMU Project	P1 Task #2 — Disk Scheduler	1h	Project spec
☐	44	Flagship Project	Flagship: Tokio tutorial part 2 (channels, I/O, framing, select)	1h	Tokio tutorial
☐	45	Contest Practice	Contest PDF #2: CF Round 1062 (Div. 4) — virtual, A–D	1.5h	Contest PDFs
☐	46	Career	DAAD motivation letter — final; upload documents  Deadline 2026-10-30	0.5h	Roadmap file
☐	47	CMU Lecture	L10 — Index Concurrency Control Reading: Chapter 18.10.2	1h	Video · Slides · Notes
☐	48	CMU Lecture	L11 — Sorting & Aggregations Algorithms Reading: Chapter 15.4–15.5	1h	Video · Slides · Notes
☐	49	CMU Project	P1 Task #3 — Buffer Pool Manager + page guards	4h	Project spec
☐	50	Repo	Networking: TCP/IP stack (OSI layers, TCP vs UDP) Tick: Networking → TCP/IP stack (OSI model layers, TCP vs. UDP)	0.5h	Guide
☐	51	Flagship Project	Flagship: tonic gRPC hello world; draft kv.proto + raft.proto	1h	GitHub
☐	52	Career	Submit the DAAD application  Deadline 2026-10-30	0.5h	Roadmap file

☐	53	CMU Lecture	L12 — Joins Algorithms Reading: Chapter 15.4-15.6 Tick: Joins	1h	Video · Slides · Notes
☐	54	CMU Project	P1 — concurrency + debugging (gdb, sanitizers)	1h	Project spec
☐	55	Repo	Networking: DNS (resolution process) Tick: Networking → DNS (Resolution process)	0.5h	Guide
☐	56	Flagship Project	Flagship: write design doc RFC-000 (scope, API, non-goals, architecture, test plan) Use the architecture section of the flagship guide as the starting point.	1h	
☐	57	Career	Send 1 application	0.5h	Roadmap file
☐	58	CMU Lecture	L13 — Query Execution I Reading: Chapter 15.1-15.3, 15.7	1h	Video · Slides · Notes
☐	59	CMU Project	P1 — pass all Gradescope tests + format + leak check, submit Milestone: P1 passes on Gradescope	1.5h	Project spec
☐	60	Flagship Project	Flagship: create the public repo — Cargo workspace, CI (fmt, clippy, test), license, README skeleton Crates: kv-storage, kv-raft, kv-server, kv-client, kv-sim, bench. Commit RFC-000 to docs/.	1h	
☐	61	Career	Send 1 application	0.5h	Roadmap file

Phase 4 · Indexes + execution				steps 62–76 · ≈12h	
☐	62	CMU Lecture	L14 — Query Execution II Reading: Chapter 22	1h	Video · Slides · Notes
☐	63	CMU Homework	HW3 Indexes & Filters — first half Tick: Databases → Indexing	1h	Homework
☐	64	Repo	Networking: HTTP (methods, status codes) Tick: Networking → HTTP protocol (Request methods, Status codes)	0.5h	Guide
☐	65	Flagship Project	Flagship: mini-lsm week 1 reading (memtable, SST format, merge iterators)	1h	mini-lsm
☐	66	Career	Send 1 application	0.5h	Roadmap file
☐	67	CMU Lecture	L15 — Query Planning & Optimization I Reading: Chapter 16 Tick: Database design and optimization	1h	Video · Slides · Notes
☐	68	CMU Homework	HW3 — finish + self-grade	1h	Homework · Solutions
☐	69	Repo	Networking: Routing algorithms; tick all finished Networking items Tick: Networking → Routing algorithms (Shortest path: Dijkstra, Bellman-Ford)	0.5h	Guide
☐	70	Flagship Project	Flagship: WAL — append with checksums, recover on start, tests Ties to CMU L21 (logging).	1h	
☐	71	Career	Send 1 application	0.5h	Roadmap file
☐	72	CMU Lecture	L16 — Query Planning & Optimization II Reading: Chapter 16 Tick: Materialized views (partly)	1h	Video · Slides · Notes
☐	73	CMU Project	P2 B+Tree — read the spec + Development Roadmap; Task #1 B+Tree pages	1h	Project spec
☐	74	Flagship Project	Flagship: memtable + flush to SSTable v0; Get/Put over storage	1h	mini-lsm
☐	75	Career	Send 1 application	0.5h	Roadmap file
☐	76	Review	Checkpoint review #2: tick checklist items; re-plan anything that slipped	0.5h	README checklist

Phase 5 · P2 B+Tree + concurrency				steps 77–96 · ≈24h	
☐	77	CMU Lecture	L17 — Concurrency Control Theory Reading: Chapter 18 Tick: Transactions; ACID properties	1h	Video · Slides · Notes
☐	78	CMU Lecture	L18 — Two-Phase Locking Concurrency Control Reading: Chapter 18.1-18.3, 18.9 Tick: Isolation, Isolation levels	1h	Video · Slides · Notes

☐	79	CMU Project	P2 Task #1 finish; Task #2 point search + insertion (with splits)	3h	Project spec
☐	80	Flagship Project	Flagship: gRPC server (Get/Put/Delete) + CLI client, end-to-end test	1h	GitHub
☐	81	Contest Practice	Contest PDF #3: ABC 476 — virtual, A–E	1.5h	Contest PDFs
☐	82	Career	Send 1 application	0.5h	Roadmap file
☐	83	CMU Lecture	L19 — Timestamp Ordering Concurrency Control Reading: Chapter 18.5-18.6	1h	Video · Slides · Notes
☐	84	CMU Lecture	L20 — Multi-Version Concurrency Control Reading: Chapter 18.7-18.8 Tick: Isolation levels (MVCC)	1h	Video · Slides · Notes
☐	85	CMU Project	P2 Task #2 — deletion (merge / redistribute); use the tree visualizer	3.5h	Project spec
☐	86	Repo	Version Control domain: Git guide + command reference, Bitbucket Tick: Version control System → Git; Bitbucket	1h	Guide · Command reference · Bitbucket
☐	87	Flagship Project	Flagship: Raft types + RequestVote/AppendEntries protos; node state machine skeleton Follow Figure 2 field by field.	1h	Raft paper
☐	88	Career	Send 1 application	0.5h	Roadmap file
☐	89	CMU Lecture	L21 — Database Logging Reading: Chapter 19.1-19.8 Tick: Atomicity, Durability	1h	Video · Slides · Notes
☐	90	CMU Project	P2 Task #3 — Index Iterator	1.5h	Project spec
☐	91	Flagship Project	Flagship: leader election with randomized timeouts + unit tests	1h	Open
☐	92	Career	Send 1 application	0.5h	Roadmap file
☐	93	CMU Lecture	L22 — Database Recovery Reading: Chapter 19.1-19.9 Tick: Durability; Transaction Control	1h	Video · Slides · Notes
☐	94	CMU Project	P2 Task #4 — Concurrency control (latch crabbing)	1.5h	Project spec
☐	95	Repo	System Design: Database Design and Optimization guide (ties to CMU L15–L16) Tick: System Design → Database design and optimization	0.5h	Guide
☐	96	Flagship Project	Flagship: turmoil simulation harness + first election test under random delays/partitions	1h	GitHub

Phase 6 · Distributed DBs + wrap-up

steps 97–104 · ≈8h

☐	97	CMU Lecture	L23 — Distributed Database Systems I Reading: Chapter 20.4-20.5, 21, 23.1-23.4 Tick: Partitioning; NoSQL databases → Types	1h	Video · Slides · Notes
☐	98	CMU Homework	HW4 Execution & Planning — solve + self-grade	1.5h	Homework · Solutions
☐	99	Flagship Project	Flagship: log replication — first AppendEntries path	1h	Raft paper
☐	100	Career	Send 1 application	0.5h	Roadmap file
☐	101	CMU Lecture	L24 — Distributed Database Systems II Reading: Chapter 20.7, 22.9 Tick: NoSQL databases → Examples	1h	Video · Slides · Notes
☐	102	CMU Project	P2 — pass Gradescope + leak check, submit Milestone: P2 passes on Gradescope	1.5h	Project spec
☐	103	Flagship Project	Flagship: M0 review — update RFC, first progress post, set M1 issues Mark M0 Done in the Project milestones database, then continue with M1.	1h	
☐	104	Review	Final review: tick all finished repo checklist items; move on to Phase 7	0.5h	README checklist

Phase 7 · Next

steps 105–138 · ≈53h

☐	105	CMU Project	CMU P3 Query Execution (Tasks 1–2) + HW5 Transactions	1h	Project spec
--------------------------	-----	-------------	---	----	------------------------------

☐	106	Repo	Languages: Programming paradigms (imperative, declarative, functional, OOP, procedural, event-driven, aspect-oriented) Tick: Programming Languages and Concepts → Programming paradigms (7 items); Functional vs. Imperative vs. Declarative	1h	Guide
☐	107	Repo	Languages: Object-oriented programming (encapsulation, inheritance, polymorphism, abstraction, composition vs inheritance, overriding vs overloading) Tick: Programming Languages and Concepts → Object-oriented programming (6 items)	1h	Guide
☐	108	Repo	Languages: Functional programming (lambdas, higher-order functions, closures, pure functions, referential transparency) + Lambda calculus Tick: Programming Languages and Concepts → Functional programming (5 items); Lambda calculus	1h	Guide
☐	109	Repo	Languages: Type systems (static vs dynamic, strong vs weak, nominal vs structural) Tick: Programming Languages and Concepts → Type systems (3 items)	0.5h	Guide
☐	110	Repo	Languages: Error handling (exceptions vs error codes, exception safety, propagation, recovery) Tick: Programming Languages and Concepts → Error handling (5 items)	0.5h	Guide
☐	111	CMU Project	CMU P3 finish + P4 Concurrency Control start	1h	Project spec
☐	112	Repo	Languages: Memory management + garbage collection (stack vs heap, manual vs automatic, tracing vs reference counting, mark-and-sweep, generational GC) Tick: Programming Languages and Concepts → Memory management (3 items); Garbage collection (2 items)	1h	Guide
☐	113	Repo	Languages: Memory layout (stack/heap, data/code segments) + Virtual memory (demand paging, page replacement) Tick: Programming Languages and Concepts → Memory layout (2 items); Virtual Memory (2 items)	1h	Guide · Virtual memory guide
☐	114	Repo	Languages: Concurrency (threads vs processes, sync primitives, thread safety, deadlocks, races, parallelism vs concurrency) Tick: Programming Languages and Concepts → Concurrency (7 items)	1h	Guide
☐	115	Repo	Languages: Asynchronous programming (callbacks, promises, futures, coroutines) Tick: Programming Languages and Concepts → Asynchronous programming (4 items)	1h	Guide
☐	116	Repo	Languages: Network programming (sockets, client-server, TCP/UDP in code, RPC) Tick: Programming Languages and Concepts → Networking (4 items)	1h	Guide
☐	117	Repo	Languages: Recursion (tail, mutual, anonymous) + Regular expressions Tick: Programming Languages and Concepts → Recursion (3 items); Regular expressions	1h	Guide · Regex guide
☐	118	Repo	Design patterns: Creational patterns Tick: Programming Languages and Concepts → Design patterns → Creational	1h	Guide
☐	119	Repo	Design patterns: Structural patterns Tick: Programming Languages and Concepts → Design patterns → Structural	1h	Guide
☐	120	Repo	Design patterns: Behavioral patterns; tick all finished Programming Languages and Concepts items Tick: Programming Languages and Concepts → Design patterns → Behavioral (also System Design → Design patterns)	1h	Guide
☐	121	CMU Project	CMU P4 finish + HW6 Recovery + L25 final review	1h	Homework
☐	122	Repo	Architecture: Client-server architecture (basics, communication protocols) Tick: System Architecture → Client-server architecture (Basics, Communication protocols)	0.5h	Guide
☐	123	Repo	Architecture: RESTful architecture (guide + reference) Tick: System Architecture → RESTful architecture	1h	Guide · Reference
☐	124	Repo	Architecture: Layered architecture (presentation, business logic, data access, cross-cutting) Tick: System Architecture → Layered Architecture (4 items)	0.5h	Guide
☐	125	Repo	Architecture: Service-Oriented Architecture (principles, advantages, challenges) Tick: System Architecture → SOA (3 items)	0.5h	Guide

☐	126	Repo	Architecture: Message queuing (basics, use cases, RabbitMQ vs Kafka) Tick: System Architecture → Message Queuing (3 items)	1h	Guide
☐	127	Repo	Architecture: Event-driven architecture (components, advantages, Kafka) Tick: System Architecture → Event-Driven Architecture (4 items)	1h	Guide
☐	128	Repo	Architecture: Microservices (guide + 76-question reference) Tick: System Architecture → Microservices; System Design → Microservices architecture	1.5h	Guide · Reference & 76 Q&A
☐	129	Repo	Architecture: Caching strategies (cache-aside, write-through, write-behind, stampede prevention); tick all finished System Architecture items Tick: System Architecture → Caching strategies (4 items)	1h	Guide
☐	130	Repo	System Design: Scalability guide Tick: System Design → Scalability	1h	Guide
☐	131	Repo	System Design: Distributed systems guide Pairs well with the flagship project (Raft, replication). Tick: System Design → Distributed systems	1h	Guide
☐	132	Repo	System Design: Object-oriented design principles; tick all finished System Design items Tick: System Design → Object-oriented design principles	1h	Guide
☐	133	Contest Practice	Contest practice — 2 virtual contests a week from the Contest-Practice PDFs (ongoing)		Contest PDFs
☐	134	Career	Mock interviews — 1 system design + 1 behavioral (STAR) a week		Guide
☐	135	Review	Repo: final checklist sweep (domains 3–8 and 10)		README checklist
☐	136	Career	Re-run the application tracker; follow up on all applications		Roadmap file
☐	137	Career	ETH Zurich Excellence Scholarship (ESOP) — apply (window 1–30 Nov)  Deadline 2026-11-30 Hard deadline: do it inside the window even if earlier steps are not finished.		Roadmap file
☐	138	Career	Keep applying: 2 applications a week from the tracker (ongoing)		Roadmap file

Flagship Project — Raft KV Store in Rust

What: a public, Raft-replicated, sharded key-value store in Rust, like a small etcd or TiKV. It has:

- a gRPC API and a Go client;
- fault-injection tests and benchmarks;
- Docker + Kubernetes deployment.

Why: it goes at the top of every CV version and covers what CMU 15-445 and GSoC 2026 don't show: distributed systems, backend services, Docker/Kubernetes, observability and measured performance.

Time: there are no fixed dates, because the time available during military service varies. Milestone **M0** is 20 short steps woven into the [step-by-step study plan](#). After step 104, work through **M1** → **M8 in order**, alternating with the Phase 7 steps. Use Friday, the one day off, for the coding-heavy milestones. The total is about 240 hours.

1. Why this project (and not more CMU or GSoC)

- **CMU projects must stay private** (the course's academic-integrity rule), so no recruiter can see them. They also cover a single-node database only: no replication, no consensus, no cloud.
- **GSoC shows Rust desktop, FFI and IPC work**, but not services, distributed systems, Kubernetes, load tests or production metrics.
- **The job ads the CVs were compared with** (Canonical, Cloudflare, IMC) asked for distributed systems, cloud, containers/Kubernetes, Go and performance. This project adds all of them truthfully.
- **It builds on CMU instead of repeating it:**
 - lectures 3–7 and 21–22 (storage, WAL, recovery) become the storage layer;
 - lectures 17–24 (concurrency control, distributed databases) become the theory behind it.

Target CV bullet. Fill in the real numbers at the end:

Built a Raft-replicated, sharded key-value store in Rust (own Raft implementation, LSM storage, gRPC). Verified linearizable under crashes and network partitions with Maelstrom/Porcupine. X ops/s and p99 Y ms on a 5-node cluster. Deployed with Docker, Helm and Prometheus/Grafana.

2. Rules that make it impressive

1. **Write your own Raft.** Use raft-rs or openraft only to compare against. Your own consensus code is what interviewers ask about.
2. **Public repo from the first commit:**
3. small, clean commits;
4. one issue per task;
5. a design doc (RFC) for every big decision.
6. **Every claim has proof:** a test, a benchmark or a graph.
7. **Deterministic simulation testing** with turmoil:
8. run thousands of randomized cluster schedules with injected faults;
9. replay any failure from its seed.
10. **Linearizability checked under faults** with Maelstrom (Jepsen) and Porcupine. Publish the fault matrix.
11. **An honest README:**
12. what works, the known limits and the benchmarks;
13. how to run it with one command.
14. **Ship in milestones.** Each milestone ends with a tag, a short post and an updated design doc.

3. What to know before you start

Topic	Why	Resource
Raft consensus	The core: election, replication, safety (Figure 2)	Raft paper · raft.github.io + visualization · Students' Guide to Raft
Distributed-systems course	Lab structure and the classic Raft bugs	MIT 6.5840 · Raft lab
Replication, partitioning, consistency	The vocabulary and trade-offs interviewers use	DDIA ch. 5, 6, 9 · Jepsen consistency models
Async Rust	Networking, timers, channels, tasks	Tokio tutorial
gRPC + protobuf	Client API and Raft RPCs	tonic
LSM storage engine	WAL, memtable, SSTables, compaction	mini-lsm · CMU lectures 3–7, 21–22
Testing distributed systems	Correctness under faults	turmoil · madsim · Maelstrom · Gossip Glomers · Porcupine · toxiproxy
Ops and cloud	Deploying and observing a cluster	kind · Helm · Prometheus · OpenTelemetry Rust
Performance	Credible benchmarks and profiling	go-ycsb · etcd performance · cargo flamegraph
Go basics	The client SDK	A Tour of Go

Topic	Why	Resource
Reference implementations (read, don't copy)	How production systems structure Raft	raft-rs · openraft · etcd raft · talent-plan

4. Architecture

```

clients (CLI, Go SDK, Rust client)
  | gRPC: Get / Put / Delete / Scan / CAS
  v
kv-server — shard router (key range → Raft group)
  |
  v
Raft groups (3–5 replicas each)
  leader election · log replication · ReadIndex reads
  snapshots · membership change
  | apply committed entries
  v
kv-storage: WAL → memtable → SSTables (LSM), compaction

ops: Docker image · docker compose · Helm chart (StatefulSet)
     Prometheus metrics · Grafana dashboard · OpenTelemetry traces

```

Cargo workspace: kv-storage, kv-raft, kv-server, kv-client, kv-sim (deterministic simulation tests), bench. The Go SDK lives in clients/go.

Design decisions to write up as RFCs:

- Raft vs Paxos;
- ReadIndex vs lease reads;
- LSM vs B+tree (and the write amplification it costs);
- range vs hash sharding;
- joint consensus vs single-server membership change;
- exactly-once client retries.

5. Milestones (in order, no dates)

#	Milestone	Est. hours	Done when
M0	Prep + skeleton: papers, Tokio/tonic, RFC-000, repo + CI, WAL + memtable, gRPC Get/Put, Raft election in simulation (the 20 flagship steps of the study plan)	20	CI green; RFC-000 in docs/; election test passes under turmoil with random delays
M1	Storage engine (WAL with checksums, memtable, SSTables + bloom filters, compaction v0, crash recovery) + gRPC Get/Put/Delete/Scan + CLI + criterion benchmarks	24	kill -9 mid-write never loses an acknowledged write; first benchmark numbers committed
M2	Raft core: election, log replication, commit, persistence, conflict handling; deterministic simulation with partitions, drops, reordering, crashes	36	Every Figure 2 rule has a test; 1000 seeded random runs pass; any failure replays from its seed
M3	Replicated KV: state machine, ReadIndex reads, client IDs for exactly-once retries, snapshots + InstallSnapshot, Maelstrom adapter	24	Maelstrom lin-kv passes on 5 nodes, including partitions
M4	Faults: docker + toxiproxy + tc netem (partitions, latency, loss, crashes, disk-full); history recorder + Porcupine	24	Fault matrix in the README, every scenario linearizable; each bug found has a regression test
M5	Scale-out: multi-Raft per key range, shard router, membership change, shard split/move, rebalancing	36	Add/remove a node and move a shard under load with no lost or duplicated writes
M6	Ops: small Docker image, compose 3/5-node clusters, Helm chart on kind, Prometheus metrics, Grafana dashboard, tracing	24	helm install gives a healthy 3-node cluster; killing the leader shows a failover on the dashboard
M7	Performance + Go SDK: YCSB-style workloads, batching/pipelining, flamegraph tuning, comparison with etcd	24	Benchmark report with ops/s and p50/p99 for 3 workloads, before and after tuning, plus methodology
M8	Ship v1.0: final README, architecture doc + RFCs, 2 blog posts, 3-minute demo video, CV/LinkedIn update, Show HN / r/rust	24	v1.0 tagged; all three CVs rebuilt with real numbers; post published; GSoC mentors asked for feedback

6. How to present it to companies

- **README:**
 - one-line pitch, architecture diagram, one-command quick start (docker compose up);
 - features table;
 - **fault-test matrix**, **benchmark charts** (ops/s, p50/p99, against etcd) and **known limits**.
- **Two blog posts:**
 1. "Building Raft in Rust: the bugs deterministic simulation found"

2. "Benchmarking my KV store against etcd"

- **3-minute demo video:** kill the leader live; the cluster keeps serving and Grafana shows the failover.
- **CV:** the first item under Projects in all three versions:
 - General SWE: numbers + the Go SDK;
 - Systems: Raft and simulation testing;
 - Embedded: the WAL, crash recovery and the latency budget.
- **Interview story (STAR):** the hardest bug, a trade-off you changed your mind on, and what you'd do differently.
- **Share it:** a LinkedIn post at v1.0, r/rust, Hacker News "Show HN", and a message to the GSoC mentors asking for feedback or a referral.

7. Routine

- **Every session:** a small commit, then tick the step.
- **Every week:** update the design doc, add a note to docs/Log.md, run the full simulation suite.
- **Every milestone:** tag a release, write a short public post, update the CV bullet.